

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



LABORATORIO 08

MEXI BANCO
SISTEMAS DISTRIBUIDOS

Saga orquestada y Circuit Breaker

Bitácora individual · Sistemas Distribuidos

DATOS ACADÉMICOS

Alumna	Estrella de Maria Mendoza Camacho
No. de cuenta	321132927
Profesor	Oscar Manuel Ruiz Hurtado
Grupo	2
Semestre	2027-1

1 Problema inicial: transferencia incompleta

1.1 Preparación y saldos iniciales

Se levantó Mexi Banco mediante Docker Compose y se comprobó que sus contenedores estuvieran saludables. Después se crearon dos cuentas para reproducir una transferencia que fallara entre el cargo y el abono.

Ana inició con $1,000 y Beto con $500.

```
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_ANA"
{"id":4,"clabe":"801792207087000001","titular":"Ana S08","saldo":1000.00}
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_BETO"
{"id":5,"clabe":"801792207087000002","titular":"Beto S08","saldo":500.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 1: Saldos iniciales de Ana y Beto antes de provocar el fallo.

1.2 Interrupción durante la transferencia

Se configuró una pausa de 20 segundos entre el cargo y el abono mediante la variable `PAUSA_ENTRE_CARGO_Y_ABONO_MS`.

Se inició una transferencia de $200 y, durante la pausa, se detuvieron las tres réplicas de Cuenta. El Gateway devolvió un error `504 Gateway Timeout`.

```
Invoke-WebRequest : {"timestamp":"2026-10-23T17:12:42.363+00:00","path":"/api/trans
ferencias","status":504,"error":"Gateway Timeout","requestId":"882bff53-180"}
En línea: 1 Carácter: 1
+ Invoke-WebRequest -UseBasicParsing `
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWeb
   Request) [Invoke-WebRequest], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.C
   ommands.InvokeWebRequestCommand
```

Figura 2: Error HTTP 504 al interrumpir la transferencia.

Después de recuperar Cuenta, se consultaron nuevamente los saldos.

```
PS C:\Users\Billie Jean\mexi-banco-s08> $CLABE_ANA = "801792207087000001"
PS C:\Users\Billie Jean\mexi-banco-s08> $CLABE_BETO = "801792207087000002"
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -i "http://localhost:8080/api/cuentas/$CLABE_ANA"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 9cdc9ca9c22b
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:40:31 GMT

{"id":4,"clabe":"801792207087000001","titular":"Ana S08","saldo":800.00}
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -i "http://localhost:8080/api/cuentas/$CLABE_BETO"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 54180762a723
Content-Type: application/json
Date: Fri, 23 Oct 2026 17:40:37 GMT

{"id":5,"clabe":"801792207087000002","titular":"Beto S08","saldo":500.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 3: Ana quedó con $800 y Beto con $500 después de la interrupción.

Cuenta	Antes	Después
Ana	$1,000	$800
Beto	$500	$500
Total	$1,500	$1,300

Resultado

El cargo de $200 se confirmó, pero el abono no se realizó. Esto ocurre porque cada microservicio confirma sus operaciones por separado y no existe una única transacción que abarque ambos cambios.

2 Implementación de la saga orquestada

2.1 Funcionamiento de la saga

Para resolver el problema se modificó `TransferenciaService.java`, que actúa como orquestador de la transferencia.

La operación sigue estos pasos:

1. Consultar las cuentas de origen y destino.
2. Registrar la transferencia como `INICIADA`.
3. Descontar el monto y guardar `CARGADA`.
4. Realizar el abono y guardar `ABONADA`.
5. Registrar los movimientos y notificaciones.
6. Finalizar en `COMPLETADA`.

Si falla el abono, el servicio intenta devolver el monto a la cuenta de origen. Cuando la devolución funciona, registra `COMPENSADA`; si también falla, registra `COMPENSACION_PENDIENTE`.

2.2 Diagrama de estados

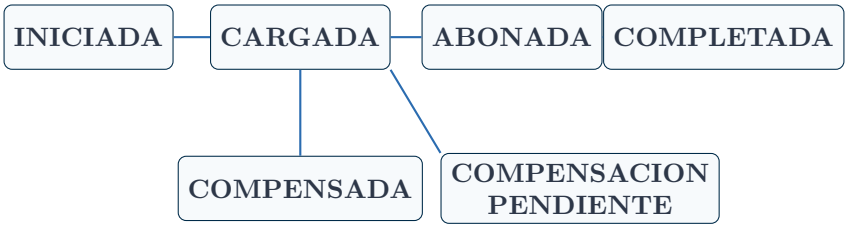


Figura 4: Diagrama de estados de la saga implementada.

Antes de realizar el cargo no hace falta devolver dinero. Después del cargo, si falla el abono, se ejecuta la compensación. Si la devolución no puede completarse, queda registrada como pendiente.

El código actual no realiza automáticamente una compensación cuando una operación posterior a un abono exitoso falla; esa situación permanece como una limitación de esta implementación.

3 Cambios principales en el código

3.1 Estados persistentes

En `Transferencia.java` se añadió un enumerado con los estados de la saga:

```
INICIADA
CARGADA
ABONADA
COMPLETADA
COMPENSADA
COMPENSACION_PENDIENTE
```

Se utilizó `@Enumerated(EnumType.STRING)` para almacenar el estado como texto en PostgreSQL.

En `TransferenciaService.java` se incorporó `saveAndFlush()` para registrar los avances de la transferencia en cada etapa.

3.2 Compensación ante un fallo

Se agregó un bloque de control de errores alrededor del abono. Cuando el abono falla, se intenta devolver el monto a la cuenta de origen utilizando el método `cuentas.abonar()`.

Si la devolución funciona, se registra un movimiento de tipo `DEPOSITO` con una referencia de compensación y se establece el estado `COMPENSADA`.

En cambio, si también falla la devolución, se guarda `COMPENSACION_PENDIENTE`.

Para provocar el primer escenario se utilizó `FALLAR_ABONO_CLABE`, variable que simula una falla al intentar depositar en la CLABE indicada.

3.3 Circuit Breaker

Se agregó la dependencia de Resilience4j en `transferencia/pom.xml`.

Después se modificó `CuentaCliente.java` para proteger las operaciones `consultar()`, `cargar()` y `abonar()` mediante un circuito compartido llamado `cuenta`.

En `application.yml` se configuró una ventana de cuatro llamadas, un umbral de fallos del 50 %, una espera de 45 segundos y dos solicitudes de prueba en estado semiabierto.

4 Prueba de compensación exitosa

4.1 Preparación

Se crearon dos cuentas nuevas para evitar modificar las utilizadas en la demostración inicial: Ana Saga con $1,000 y Beto Saga con $500.

```
PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== ANA SAGA ==="
=== ANA SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_ANA_SAGA"
{"id":6,"clabe":"811792778770000001","titular":"Ana Saga","saldo":1000.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== BETO SAGA ==="
=== BETO SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_BETO_SAGA"
{"id":7,"clabe":"811792778770000002","titular":"Beto Saga","saldo":500.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 5: Cuentas de prueba para comprobar la compensación de la saga.

4.2 Fallo simulado y devolución

Se configuró `FALLAR_ABONO_CLABE` con la CLABE de Beto Saga y se solicitó una transferencia de $200.

El servicio devolvió `HTTP 409 Conflict`. Al consultar los saldos, Ana conservaba $1,000 y Beto $500.

Esto comprobó que el cargo fue compensado y que no se produjo un abono indebido.

```
PS C:\Users\Billie Jean\mexi-banco-s08> $CLABE_ANA_SAGA = "81179277877000001"
PS C:\Users\Billie Jean\mexi-banco-s08> $CLABE_BETO_SAGA = "81179277877000002"
PS C:\Users\Billie Jean\mexi-banco-s08>
PS C:\Users\Billie Jean\mexi-banco-s08> $datos = @{
>>     claveOrigen = $CLABE_ANA_SAGA
>>     claveDestino = $CLABE_BETO_SAGA
>>     monto = 200
>> } | ConvertTo-Json
PS C:\Users\Billie Jean\mexi-banco-s08>
PS C:\Users\Billie Jean\mexi-banco-s08> try {
>>     Invoke-WebRequest -UseBasicParsing `
>>         -Uri "http://localhost:8080/api/transferencias" `
>>         -Method POST `
>>         -ContentType "application/json" `
>>         -Body $datos `
>>         -ErrorAction Stop
>> }
>> catch {
>>     Write-Host "HTTP:" ([int]$_.Exception.Response.StatusCode)
>>     Write-Host $_.ErrorDetails.Message
>> }
HTTP: 409

PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== ANA DESPUES DE LA SAGA ==="
=== ANA DESPUES DE LA SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_ANA_SAGA"
{"id":6,"clabe":"81179277877000001","titular":"Ana Saga","saldo":1000.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== BETO DESPUES DE LA SAGA ==="
=== BETO DESPUES DE LA SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/$CLABE_BETO_SAGA"
{"id":7,"clabe":"81179277877000002","titular":"Beto Saga","saldo":500.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 6: Respuesta HTTP 409 y saldos conservados después de la compensación.

4.3 Comprobación en PostgreSQL

Se consultó directamente la tabla `transferencia` mediante `psql`.

La operación quedó registrada con estado `COMPENSADA` y monto de $200.

```
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec -T db psql -U mexibanco -d transferencia -c "SELECT id, estado, monto FROM transferencia ORDER BY id DESC LIMIT 5;"
 id |   estado    | monto
----+-------------+--------
  2 | COMPENSADA  | 200.00
  1 |             | 200.00
(2 rows)
```

Figura 7: PostgreSQL confirma el estado COMPENSADA de la transferencia.

5 Prueba de compensación pendiente

Para probar qué sucede cuando tampoco se puede devolver el dinero, se crearon otras dos cuentas: Ana Pendiente con $1,000 y Beto Pendiente con $500.

Se activó nuevamente la pausa de 20 segundos. Durante una transferencia de $200 se apagaron las tres réplicas de Cuenta.

De esta manera falló el abono y tampoco pudo realizarse la compensación.

Al consultar PostgreSQL, la transferencia con ID 3 apareció como `COMPENSACION_PENDIENTE`.

```
ferencia -c "SELECT id, estado, monto FROM transferencia ORDER BY id DESC LIMIT 5;"
 id |         estado          | monto
----+-------------------------+--------
  3 | COMPENSACION_PENDIENTE  | 200.00
  2 | COMPENSADA              | 200.00
  1 |                         | 200.00
(3 rows)

PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 8: Transferencia 3 registrada como COMPENSACION$_PENDIENTE$.

5.1 Recuperación e idempotencia

El estado pendiente permite identificar una operación que necesita atención. Una forma de terminarla sería mediante un proceso programado que revisara las transferencias pendientes y reintentara la devolución.

Sin embargo, ese reintento debe ser **idempotente**: si se ejecuta varias veces, no debe devolver el dinero más de una vez.

Para ello se puede utilizar un identificador único de compensación y comprobar si ya fue aplicada antes de repetirla.

En esta práctica se implementó el registro del estado pendiente, pero no su recuperación automática.

6 Circuit Breaker con Resilience4j

6.1 Estado CLOSED

Inicialmente se comprobó el endpoint `/actuator/circuitbreakers`.

Resilience4j mostró el circuito `cuenta` en estado `CLOSED`, sin fallos registrados. En este estado las peticiones HTTP se permiten normalmente.

```
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec transferencia printenv PAUSA_ENTRE_CARGO_Y_AB
ONO_MS
0
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec -T transferencia wget -qO- http://localhost:8
080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%"
,"slowCallRateThreshold":"100.0%","bufferedCalls":0,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"no
tPermittedCalls":0,"state":"CLOSED"}}}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 9: Estado inicial CLOSED del Circuit Breaker.

6.2 Estado OPEN

Se apagaron las tres réplicas de Cuenta y se realizaron cinco intentos de transferencia.

Los primeros cuatro intentos devolvieron `HTTP 500` y el quinto devolvió `HTTP 503`.

Al consultar el Circuit Breaker se observaron cuatro llamadas fallidas, una llamada rechazada y una tasa de fallos del 100 %.

El circuito cambió a `OPEN` y dejó de permitir nuevas llamadas al servicio.

```
Intento 1 - HTTP 500
Intento 2 - HTTP 500
Intento 3 - HTTP 500
Intento 4 - HTTP 500
Intento 5 - HTTP 503
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec -T transferencia wget -qO- http://localhost:8
080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"100.0%","slowCallRate":"0.0%","failureRateThreshold":"50.0%"
,"slowCallRateThreshold":"100.0%","bufferedCalls":4,"failedCalls":4,"slowCalls":0,"slowFailedCalls":0,"no
tPermittedCalls":1,"state":"OPEN"}}}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 10: Circuit Breaker en OPEN: cuatro fallos y una llamada rechazada.

6.3 Estado HALF_OPEN

Después de los 45 segundos configurados, el circuito pasó automáticamente a `HALF_OPEN`.

Este estado permite realizar un número limitado de solicitudes para comprobar si Cuenta puede responder nuevamente.

```
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec -T transferencia wget -qO- http://localhost:8
080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%"
,"slowCallRateThreshold":"100.0%","bufferedCalls":0,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"no
tPermittedCalls":0,"state":"HALF_OPEN"}}}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 11: Transición del Circuit Breaker al estado HALF$_O PEN$.

6.4 Recuperación a CLOSED

Se recuperaron las tres réplicas de Cuenta y se verificó su registro en Eureka.

Tras completar las llamadas de prueba exitosas, el Circuit Breaker regresó a `CLOSED`.

El endpoint mostró dos llamadas registradas y ninguna fallida.

```
PS C:\Users\Billie Jean\mexi-banco-s08> docker compose exec -T transferencia wget -qO- http://localhost:8
080/actuator/circuitbreakers
{"circuitBreakers":{"cuenta":{"failureRate":"-1.0%","slowCallRate":"-1.0%","failureRateThreshold":"50.0%"
,"slowCallRateThreshold":"100.0%","bufferedCalls":2,"failedCalls":0,"slowCalls":0,"slowFailedCalls":0,"no
tPermittedCalls":0,"state":"CLOSED"}}}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 12: El Circuit Breaker regresa a CLOSED sin fallos.

Finalmente, se comprobó una transferencia válida de $1 entre Ana Saga y Beto Saga.

Los saldos finales fueron $999 para Ana y $501 para Beto.

```
PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== ANA SAGA ==="
=== ANA SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/811792778770000001
"
{"id":6,"clabe":"811792778770000001","titular":"Ana Saga","saldo":999.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
PS C:\Users\Billie Jean\mexi-banco-s08> Write-Host "=== BETO SAGA ==="
=== BETO SAGA ===
PS C:\Users\Billie Jean\mexi-banco-s08> curl.exe -s "http://localhost:8080/api/cuentas/811792778770000002
"
{"id":7,"clabe":"811792778770000002","titular":"Beto Saga","saldo":501.00}
PS C:\Users\Billie Jean\mexi-banco-s08>
```

Figura 13: Saldos finales después de recuperar el Circuit Breaker.

Resultado del Circuit Breaker

Se observaron los estados CLOSED, OPEN y HALF_OPEN. Cuando Cuenta estuvo fuera de servicio, el circuito dejó de permitir nuevas llamadas. Después de recuperarlo y realizar llamadas exitosas, volvió a CLOSED.

7 Limitación de aislamiento de la saga

Aunque la saga permite compensar una operación fallida, no ofrece el mismo aislamiento que una transacción realizada dentro de una sola base de datos.

Por ejemplo, entre el cargo y la compensación, una persona que consulte la cuenta de origen puede observar un saldo de $800, aunque la transferencia de $200 todavía no haya terminado.

Si posteriormente la compensación funciona, el saldo regresa a $1,000. Esto significa que otros servicios pueden observar estados intermedios mientras la saga está en ejecución.

Por lo tanto, la compensación ayuda a recuperar la consistencia, pero no evita que esos cambios temporales sean visibles.